

Name: Kanika Saini

SAP ID: 590034098

Program: B.Tech CSE (B16)

Question 1:

Program: Find Horizontal Distance in Projectile Motion

Step 1: Problem Understanding

Input: Horizontal velocity (u) in m/s

Time (t) in seconds

Output: Horizontal distance (d) in meters

Formula: $d = u \times t$

Step 2: Test Cases Table

Test Case	Inputs (u,t)	Expected Output	Actual Output	Result
1.	20 , 5	100m	100m	Pass
2.	15 , 8	120m	120m	Pass
3.	30 , 2	60m	60m	Pass

Step 3: Algorithm

1. Start.
 2. Input horizontal velocity and time.
 3. Calculate distance = velocity $\times$ time.
 4. Display the distance.
 5. Stop.
-

Step 4: C Program-

Make "dist.c" file first and write this code-

```
#include<stdio.h>

int main()
{
    float u,t,d;

    printf("Enter horizontal velocity: ");
    scanf("%f",&u);

    printf("Enter time: ");
    scanf("%f",&t);

    d=u*t;
```

```
printf("Horizontal Distance = %.2f m",d);  
  
return 0;  
  
}
```

Step 5: Evaluation

The program was executed using the three test cases prepared in Step 2. In all three cases, the actual output matched the expected output. Therefore, all test cases passed successfully, confirming that the program correctly calculates the horizontal distance in projectile motion.

Step 6: Compilation and execution

```
(kali㉿kali)-[~/Desktop]  
$ cc dist.c  
  
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
Enter horizontal velocity: 20  
Enter time: 5  
Horizontal Distance = 100.00 m
```

```
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
Enter horizontal velocity: 15  
Enter time: 8  
Horizontal Distance = 120.00 m
```

```
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
Enter horizontal velocity: 30  
Enter time: 2  
Horizontal Distance = 60.00 m
```

Question 2:

Program: Find Length of Wire

Step 1: Problem Understanding

Input: Resistance (R) , Area (A) , Resistivity (ρ)

Output: Length (L)

Formula: $L = (R \times A) / \rho$

Step 2: Test Cases Table

Test Case	Inputs (R,A,rho)	Expected Output	Actual Output	Result
1.	10, 2, 5	4	4	Pass
2.	20, 3, 2	30	30	Pass
3.	15, 4 , 5	12	12	Pass

Step 3: Algorithm

1. Start.
 2. Read resistance, area and resistivity.
 3. Calculate length = (Resistance $\times$ Area)/Resistivity.
 4. Display the length.
 5. Stop.
-

Step 4: C Program

Make "length.c" file first and write this code-

```
#include<stdio.h>

int main()
{
    float R,A,rho,L;

    printf("Enter resistance: ");
    scanf("%f",&R);

    printf("Enter area: ");
    scanf("%f",&A);

    printf("Enter resistivity: ");
    scanf("%f",&rho);

    L=(R*A)/rho;

    printf("Length of Wire = %.2f",L);
    return 0;
```

}

Step 5: Evaluation

The program was tested with the three test cases prepared in Step 2. The actual output matched the expected output for every test case. Hence, all test cases passed, proving that the program correctly calculates the length of the wire.

Step 6: Compilation and Execution-

```
(kali㉿kali)-[~/Desktop]
$ cc length.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter resistance: 10
Enter area: 2
Enter resistivity: 5
Length of Wire = 4.00
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter resistance: 20
Enter area: 3
Enter resistivity: 2
Length of Wire = 30.00
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter resistance: 15
Enter area: 4
Enter resistivity: 5
Length of Wire = 12.00
```

Question 3:

Program: Find Nuclear Binding Energy

Step 1: Problem Understanding

Input: Mass defect (m), Speed of light (c)

Output: Binding energy (E)

Formula: $E=mc^2$

Step 2: Test Cases Table

Test Case	Inputs (m,c)	Expected Output	Actual Output	Result
1.	2 , 3	18	18	Pass
2.	5 , 4	80	80	Pass
3.	1 , 5 , 5	37.5	37.5	Pass

Step 3: Algorithm

1. Start.
 2. Read mass defect and speed of light.
 3. Calculate binding energy = mass defect × speed of light × speed of light.
 4. Display the binding energy.
 5. Stop.
-

Step 4: C Program-

Make “energy.c” file first and write this code-

```
#include<stdio.h>

int main()
{
    float m,c,E;

    printf("Enter mass defect: ");
    scanf("%f",&m);

    printf("Enter speed of light: ");
    scanf("%f",&c);

     $E=m*c*c$ ;

    printf("Binding Energy = %.2f",E);

    return 0;
}
```

Step 5: Evaluation

The program was evaluated using the three test cases prepared in Step 2. For each test case, the actual output was the same as the expected output. Therefore, all test cases passed successfully, confirming that the program correctly calculates the nuclear binding energy.

Step 6 : Compilation and Execution-

```
(kali㉿kali)-[~/Desktop]
$ cc energy.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter mass defect: 2
Enter speed of light: 3
Binding Energy = 18.00
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter mass defect: 5
Enter speed of light: 4
Binding Energy = 80.00
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter mass defect: 1.5
Enter speed of light: 5
Binding Energy = 37.50
```